

NOXTILL — COMPLETE SYSTEM SPECIFICATION (Developer Edition)

All 17 Modules • Step by Step • Backend + Frontend • Research-Backed • Nothing Missing

How to use: Save as `SPEC.md` in the project root. This is the single source of truth. Build order for a 2-month launch is at the end (Section E).

SECTION A — PRODUCT DEFINITION

A1. Identity

Item	Value
Name	Noxtill (nox = night, till = shop counter)
Tagline	"Close your day in one tap."
Signature feature	Nightly Close — one message every night (default 10 PM, business's local time) with: today's sales, today's profit, new reviews + avg rating, tomorrow's bookings, credit recovered, low-stock alerts, link to full day view
Core principle	Automation through connection — every module auto-triggers the next: booking → reminder → visit → sale → (stock + profit + customer update) → review request → Nightly Close
Target users	ALL customer-facing SMBs worldwide: restaurants, cafes, bakeries, clinics, dentists, salons, barbershops, gyms, lawyers, accountants, retail shops, e-commerce, plumbers, electricians, hotels, schools, photographers, agencies, software companies, wholesalers + ANY unlisted type via AI setup

A2. Market research (validated during planning)

- WhatsApp API pricing (Pakistan example):** Marketing message ≈ PKR 13.20, Utility ≈ PKR 2.79, replies within the 24-hour customer-service window = FREE. Utility rates rose in April 2026 — always classify transactional messages (confirmations, reminders, receipts, credit reminders) as **Utility** (≈5x cheaper) and promotions as **Marketing**.
- Meta access:** integrate **Meta Cloud API directly** (skip BSP monthly fees ~PKR 8,000–15,000/mo).

- **Competitor landscape (why Noxtill wins = integration):**
 - Podium / Birdeye → reviews only, no sales
 - Square / Lightspeed / Tillora → POS only, no reputation
 - Calendly / Fresha → bookings only
 - Zoho / Odoo / QuickBooks → powerful but too complex for shop owners, WhatsApp-blind
 - Khatabook / OkCredit / ShopBook (funded, Sri Lanka) → credit ledger only
 - Owner.com (\$178M raised) → restaurants only
 - **Nobody combines all of it + Nightly Close + AI + WhatsApp-first + offline. That is the product.**
- **Google policy:** review **gating is prohibited** — same link for every customer; routing happens by the customer's own star choice (compliant design below).
- **Unit economics:** API cost $\approx$ 25–30% of revenue at scale; margins 70%+; message costs must be baked into plan quotas.

A3. Pricing plans (enforce quotas everywhere)

Plan	Price (USD ref)	Includes	Messages/mo	Users
Basic	~\$10/mo	Reviews, routing, inbox, AI replies, QR, widget	150	1
Growth	~\$25/mo	+ Bookings, orders, billing, invoices, credit ledger, CRM	600	3
Business	~\$50/mo	+ P&L, inventory, staff, AI Assistant, what-if, analytics, reports	2,000	10
Enterprise	~\$100+/mo	+ Multi-location, branch AI, priority support	Custom	Unlimited

- 14-day free trial, no card. Localized pricing per market. Overage → upgrade prompt.
- Payments: Stripe (global) + pluggable regional gateways (JazzCash/Easypaisa, Razorpay, Mercado Pago, Paystack) — **configuration, never code.**

A4. Brand & design system

- Colors: paper (□ #FAF7F0) · ledger green (■ #0C4B3B) (primary) · ink (■ #182420) · marigold (■ #E8A93C) (accent) · muted red (■ #B94A3D) (**credit balances only**)

- Fonts: Bricolage Grotesque (display) · Instrument Sans (body) · Noto Nastaliq Urdu / Noto Naskh Arabic (locale accents)
- Motifs: ledger-ruled dividers · 14px-radius cards · crescent-moon + ledger-line logo
- Dashboard laws: **three-second rule** (day understood at a glance) · **every number clickable** to its records · mobile-first (owners use phones)

A5. Global-first rules (every module)

- ☐ Multi-currency: ISO 4217 chosen at signup; used in ALL money displays, invoices, PDFs, AI answers
- ☐ Multi-language: all strings in translation files; launch EN, ES, PT, FR, AR, UR, HI; full RTL; AI replies in the user's/reviewer's language
- ☐ Multi-timezone: every schedule (reminders, Nightly Close) in the business's local time
- ☐ Channels by region: WhatsApp-first where dominant; SMS/email elsewhere; per-business override
- ☐ Tax: configurable label + rate (VAT/GST/sales tax)
- ☐ Privacy: GDPR baseline — consent tracking, opt-outs honored everywhere, right to erasure, full data export
- ☐ **Hard-code nothing regional**

A6. Non-negotiable engineering rules

- ☐ Every table has `business_id`; every query tenant-scoped at the data-access layer; cross-tenant access impossible (incl. exports, search index, webhooks, AI tools) — automated isolation tests required
- ☐ Every external call goes through the queue (never inside a request): retries, backoff, dead-letter, per-message delivery status
- ☐ Every webhook idempotent (store event ID; ignore repeats)
- ☐ Append-only `audit_log` for every financial mutation (actor, action, before/after, timestamp)
- ☐ RBAC: Owner / Manager / Staff (staff = own sales only; full customer export = owner only)
- ☐ Files via signed URLs, 24-hour expiry
- ☐ Daily DB backups + tested restores
- ☐ **No review gating. Ever.**
- ☐ AI: read-only in v1; numbers only from tool results; says "not found" instead of guessing
- ☐ Never message opted-out customers
- ☐ All SEO white-hat

A7. Tech stack

Layer	Choice
Backend	Node.js + NestJS
Database	PostgreSQL (single DB, multi-tenant via business_id)
Queue/scheduler	Redis + BullMQ
Web frontend	Next.js + React + Tailwind + React Query
Mobile	React Native (one codebase iOS + Android)
Search	Postgres FTS + pg_trgm → upgradable to Meilisearch/Typesense
AI	Anthropic Claude API (fast low-cost model for volume tasks)
Files	S3-compatible + Puppeteer (PDF) + exceljs (Excel)
Messaging	Meta WhatsApp Cloud API (direct) · Twilio/local SMS · SES/Postmark email
Payments	Stripe + regional gateway plugins

A8. Database tables (complete — build ALL in week 1)

`business_categories` (12 patterns, seeded) · `business_types` (200–300 rows + ai_generated flag) · `widgets` (registry) · `plans` · `businesses` (currency, timezone, locale, channel_pref, nightly_close_time, tax, plan, msg_quota/used, branding JSON, dashboard_config JSON, parent_id for branches) · `users` · `business_users` (role + commission_rule) · `customers` (phone-keyed, tags[], consent, opted_out, lifetime_spend, visit_count, last_visit_at) · `products` (kind product|service, cost_price, selling_price, stock_qty, low_stock_threshold, duration, variations JSON) · `orders` (order_no per business, order_type, table_no, status, subtotal/tax/discount/total, profit) · `order_items` (name/price/cost **snapshots**) · `payments` (method incl. credit) · `credit_entries` (kind credit|payment) · `appointments` (status incl. no_show, deposit, source) · `messages` (channel, category utility|marketing, template_key, payload, status lifecycle, scheduled_for) · `webhook_events` (provider+event_id PK) · `review_requests` (token, stars, routed_to, reminder_count) · `private_feedback` (ticket: open|assigned|resolved) · `external_reviews` (platform, external_id, reply) · `stock_movements` (purchase|sale|wastage|adjustment) · `expenses` (category, recurring) · `attendance` · `campaigns` · `competitors` · `audit_log` (append-only) · views: `v_credit_balances`, `v_daily_close`

SECTION B — THE 17 MODULES (original numbering, nothing merged)

MODULE 1 — REVIEWS & REPUTATION

Purpose: grow public rating + review count; catch unhappy customers privately.

Features (all):

- Auto review request 2h (configurable) after completed sale/appointment · manual send · QR entry
- Auto reminders day 3 + day 7; stop on response
- White-label rating page (logo, colors) — 1-5 stars, mobile-first
- **Compliant smart routing** (same link for everyone): 4-5★ → one-tap redirect to public review URL (Google default; Yelp/Facebook/Trustpilot/TripAdvisor configurable) · 1-3★ → private feedback form → never public
- Instant owner alert on every private complaint (message + dashboard)
- Complaints = tickets: open → assigned → resolved
- Unified inbox: Google (Business Profile API, free) + Facebook + others; reply from dashboard
- **AI reply drafts** in reviewer's language; owner one-tap approve (auto-mode later)
- Rating trend chart · request→review conversion stat · complaint themes
- QR code generator + print-ready counter-stand design
- Embeddable review widget (best reviews + "Powered by Noxtill" backlink)

Backend steps:

1. `review_requests` CRUD + unique token
2. Trigger hooks from sales (M3) + completed appointments (M2) + manual + QR
3. Public: `GET /r/:token` (branding) · `POST /r/:token` (stars → route)
4. Routing logic per above; save `routed_to`
5. Reminder scheduler (BullMQ)
6. Google sync worker: pull reviews → `external_reviews`; post replies
7. `POST /reviews/:id/ai-draft` (Claude, language-matched)
8. Ticket endpoints: assign, resolve
9. Public widget endpoint (cached)

Frontend screens: rating page · private feedback form · unified inbox (filters, reply, AI-draft button) · tickets list · trend chart · QR download · widget snippet generator with preview

Done test: WhatsApp request → 5★ lands on Google in 2 taps → 2★ creates ticket + instant owner alert → owner replies to Google review inside Noxtill.

MODULE 2 — BOOKINGS & APPOINTMENTS

Purpose: replace phone-and-diary; cut no-shows 30-40%; feed the review funnel.

Features (all):

- Services: name, duration, price, optional staff
- Public booking page + QR (white-label): service → staff (optional) → free slot → name + phone → confirm
- Availability = working hours + staff schedules + existing bookings (race-safe slot lock)
- Optional advance deposit via payment link
- Owner calendar: day/week views, per-staff columns, drag-reschedule, walk-in add
- Statuses: booked / confirmed / completed / no_show / cancelled
- Auto confirmation message · reminders 24h + 2h with one-tap reschedule/cancel token links (self-updating calendar)
- Completed → auto review request (M1) · no-show → recorded on customer (M7)

Backend steps: 1. services CRUD → 2. availability computation → 3. public booking endpoints + slot locking → 4. reminder jobs → 5. reschedule/cancel token links → 6. completion hook → M1 → 7. deposit hook.

Frontend screens: public booking page · calendar (staff columns, drag) · walk-in modal · services management

Done test: customer books from link → reminded → reschedules from the message → visit completes → review request fires by itself.

MODULE 3 — PRODUCTS, ORDERS & BILLING (POS-lite)

Purpose: the money core.

Features (all):

- Catalog: products AND services — name, photo, category, variations (size/color), **cost_price**, **selling_price**, stock_qty, low_stock_threshold, duration (services), CSV import

- Fast sale screen: tap items → qty → discount → tax auto → payment: cash / card / online / **credit**
- **Single-transaction sale** (atomic): order + items (cost snapshots → per-line profit frozen at sale time) + stock decrement + stock_movement + customer upsert by phone + lifetime stats update + payment OR credit entry + audit log + queue review request (2h)
- Per-business order numbering
- Order types: counter / online link / dine-in (table no.) / takeaway / delivery / quotation
- Restaurant: table-status board · QR table menu (order from table)
- E-commerce: shipping fields, payment status
- Status flow: pending → confirmed → in_progress → completed → refunded/cancelled, with auto customer status messages
- Invoice/receipt PDFs: branded, currency + tax lines; print (receipt-printer friendly) or send by WhatsApp
- Quotations → PDF → convert to order on approval
- Barcode scanner input

Backend steps: 1. products CRUD + import → 2. single-transaction sale endpoint → 3. status endpoints + messages → 4. Puppeteer invoice/receipt → 5. quotation flow → 6. public online-order endpoints → 7. table/QR menu endpoints.

Frontend screens: products list (stock badges, import) · fast sale (item grid + cart + payment buttons, phone-first) · orders board + table view · invoice preview/print/send · quotation builder · public order page + QR menu

Done test: sale under 10 seconds on a phone → receipt on customer's WhatsApp → stock, profit, customer history all moved in one transaction → credit sale opens ledger entry.

MODULE 4 — CUSTOMER CREDIT LEDGER

Purpose: digitize tab/udhaar/khata/fiado — a global differentiator incumbents ignore.

Features (all):

- Credit-type sale → per-customer ledger (keyed by phone): balance = credits – payments
- Credit screen: every debtor + balance + last entry date · total receivable on top
- Record payment (reduces balance)
- Per-customer **PDF statement** (share with customer)

- **One-tap polite reminder** — one debtor or ALL: "You have an outstanding balance of X — thank you!" (localized label: tab/udhaar/khata/fiado)
- Recoveries appear in the Nightly Close · every entry audit-logged

Backend steps: 1. credit entry/payment endpoints → 2. balances view → 3. statement PDF → 4. remind-one + remind-all (via M-Engine, Utility class).

Frontend screens: credit screen (debtor list, totals, buttons) · payment modal · statement download

Done test: credit sale → balance appears → reminder sends → payment clears → Nightly Close shows recovery.

MODULE 5 — INVENTORY

Features (all):

- Auto stock decrement on sale (wired in M3)
- Purchase entries: supplier, qty, unit cost → stock up + cost_price refreshed
- Low-stock alerts (per-product threshold) → dashboard + Nightly Close
- Wastage entries with reasons (expired/damaged) → honest profit
- Stock value at cost · supplier history per product

Backend steps: 1. purchase + wastage endpoints (write stock_movements) → 2. low-stock scan job → 3. valuation endpoint.

Frontend screens: inventory list · purchase form · wastage form · low-stock view

Done test: buy 50 → stock + cost update; waste 5 → profit reflects; low item alerts tonight.

MODULE 6 — PROFIT & LOSS ENGINE

Features (all):

- Per-product: margin %, profit/unit, units sold, total contribution, **loss-maker flags** ("Cold Coffee: 8% — review pricing")
- Best sellers + dead stock (configurable window)
- **Time analysis:** sales by hour/day/week/month ("60% of sales happen 7–10 PM"; slowest day)
- Expenses: one-off + recurring (rent, salaries, utilities, supplies), categories
- **True monthly P&L:** revenue – COGS – expenses → export for accountant/bank

- **AI what-if calculator:** "raise this price 10%?" — computed ONLY from the business's own history; ALWAYS labeled estimate, never guarantee
- AI pricing notes surfaced on profitability rows

Backend steps: 1. profitability queries → 2. time-bucket queries → 3. expenses CRUD + recurring job → 4. P&L composer → 5. what-if endpoint (Claude + history).

Frontend screens: product profitability table (flags) · time charts · best/dead lists · expenses · P&L statement · what-if calculator (sliders, "estimate" label)

Done test: owner sees real peak hours, worst item, and a true month P&L.

MODULE 7 — CUSTOMER CRM

Features (all):

- Identity = phone number; records self-build from every booking/sale/review (zero data entry)
- Profile page: header (name, VIP badge, phone, since) · stat cards (lifetime spend, visits, last visit, credit balance) · full purchase history with serving staff · reviews/complaints · owner notes (allergies, preferences)
- Tags: auto (VIP by spend threshold, New, Lapsed by absence — configurable) + manual
- Instant search: name or last phone digits
- One-tap WhatsApp from profile
- Segments + one-tap campaigns: Lapsed → win-back · VIP → exclusive offers
- Auto birthday greetings (optional offer code)
- Consent/opt-out per customer (enforced by M-Engine) · per-customer erasure

Backend steps: 1. profile aggregate endpoint → 2. nightly tag-rules job → 3. segment queries → 4. birthday job → 5. erasure endpoint.

Frontend screens: customers list (search + tag filters) · profile page (as designed) · segment view + message-all

Done test: "678" finds Ayesha; her file shows everything; Lapsed win-back sends in 2 taps.

MODULE 8 — STAFF MANAGEMENT

Features (all):

- Staff accounts + role permissions (see A6)

- Attendance check-in/out
- Sale + appointment attribution per staff
- Commission rules per staff (percent of sales or per-service amount) → automatic monthly commission report
- Performance dashboard (sales, appointments, reviews mentioning them)
- Team inbox: complaints/tasks assigned + tracked to resolution

Backend steps: 1. staff CRUD + roles → 2. attendance endpoints → 3. commission calculator + report → 4. team inbox endpoints.

Frontend screens: staff list · attendance · commissions report · team inbox

Done test: salon pays staff from the commission report; a complaint is assigned and resolved.

MODULE 9 — MARKETING & GROWTH TOOLS

Features (all):

- Broadcast campaigns: all customers or any CRM segment (festival offers) — quota, rate limiting, opt-out compliance, send report (delivered/read)
- Social auto-post: 5★ review → branded image → connected Facebook/Instagram
- Referral program: after positive review → shareable "give 10%, get 10%" code, trackable
- Competitor tracking: owner adds 3–5 rivals → weekly comparison message ("You 4.2, rival 4.5") + local-search ranking indicator
- Birthday campaigns (from M7) · QR assets + review widget as marketing surfaces

Backend steps: 1. campaign engine over M-Engine + quotas → 2. send reports → 3. social post worker → 4. referral codes + tracking → 5. competitor weekly worker.

Frontend screens: campaign builder (segment → template → preview → send → report) · referral settings · competitor screen · social settings

Done test: Eid/holiday campaign to Lapsed within quota + delivery stats; weekly rival comparison arrives.

MODULE 10 — MULTI-LOCATION & ENTERPRISE

Features (all):

- Branches under a parent business (`(parent_id)`)
- Per-branch: dashboard, staff, catalog overrides
- Combined roll-up view · side-by-side branch comparison (sales, profit, ratings, staff)
- **AI branch advisor:** explains gaps between the owner's OWN branches from their OWN data only — never external advice (never "where to open next")
- Enterprise plan enforcement (unlimited users, custom limits)

Backend steps: 1. branch scoping → 2. roll-up queries → 3. comparison endpoint → 4. branch-advisor AI tool.

Frontend screens: branch switcher · roll-up dashboard · comparison view

Done test: 3-branch chain sees combined + per-branch; advisor explains why A beats B.

MODULE 11 — WIDGET DASHBOARD + UNIVERSAL BUSINESS-TYPE SYSTEM

Features (all):

- Dashboard = grid of independent **widgets**, each with own data endpoint (25–30 at launch): `today's_sales`, `today's_profit`, `new_reviews`, `avg_rating`, `today's_bookings`, `pending_orders`, `active_tables`, `occupancy`, `hearings_today`, `fees_due`, `members_present`, `low_stock`, `credit_outstanding`, `staff_commissions`, `top_product`, `staff_sales`, ...
- 3 data structures: `(widgets)` registry · `(dashboard_templates)` per category · per-business `(dashboard_config)` JSON
- **Level 1:** 12 base patterns — food & dining · health & appointments · beauty & care · fitness & memberships · professional services · retail & e-commerce · field services · hospitality & rentals · education · software & agencies · manufacturing & wholesale · general
- **Level 2:** 200–300 named types (dentist, bakery, law firm, plumber, hotel, tuition center...) — each = a DB row (never a deploy), adjusts labels (patients/clients/members/guests) + default widgets; each type powers an SEO page (M17)
- **Level 3 — AI fallback:** "Other" + one line ("mehndi artist") → Claude returns `{pattern, widgets, labels}` JSON → dashboard generated → new type SAVED (directory grows itself worldwide)
- Owner customization: edit mode → remove × → "+ Add widget" gallery → drag-and-drop (dnd-kit/react-grid-layout) → saved
- Home-screen laws: three-second rule · every number clickable

Backend steps: 1. widget registry + per-widget endpoints → 2. templates + config APIs → 3. searchable type list → 4. AI type-mapper + save-new-type → 5. label-override system.

Frontend screens: signup type picker (search + categories + AI Other) · dashboard rendered from config · customization mode

Done test: dentist, restaurant, and AI-mapped "mehndi artist" see correct different dashboards; rearrangement persists; a new type row = a new landing page, zero deploy.

MODULE 12 — DEEP SEARCH

Features (all):

- One search bar on EVERY screen + keyboard shortcut
- Searches at once: customers (name/partial phone), orders/invoices (number/amount/item), products, bookings, reviews/complaints, credit entries, expenses, settings, help articles
- Typo-tolerant (pg_trgm) · instant as-you-type · results grouped by category · deep links to the record
- Index strictly tenant-scoped · also powers AI retrieval + CRM lookup
- Upgrade path: Postgres FTS → Meilisearch/Typesense behind the same interface

Backend steps: 1. unified search endpoint + grouping → 2. trigram indexes → 3. tenant-scope tests.

Frontend: global search bar + grouped instant results panel

Done test: typing "biry" finds Chicken Biryani, its orders, and its stock line in <150ms.

MODULE 13 — ANALYTICS (high-quality, both sides)

Owner analytics — features:

- Unified over any date range with comparisons (this vs last month): revenue, profit, orders, bookings, review scores, customer growth
- **Retention cohorts:** of last month's customers, how many returned
- Campaign performance (sent/delivered/read/redeemed) · staff performance · channel stats (delivery/read rates)
- Every metric traceable — click through to source records; all exportable; AI plain-language monthly summary

Platform analytics (Noxtill internal) — features:

- Event taxonomy instrumented from day 1: signup_started/completed · activation steps (google_connected, first_product, first_review_request, first_sale, first_nightly_close) · feature_used(module) · message_sent(channel, class) · plan_upgraded/downgraded · churn signals (no login 14d, quota unused)
- **Admin panel:** tenants list, MRR, churn %, cohort retention, usage per module, message volume/cost
- KPI definitions: Activation = first Nightly Close within 72h of signup · North-star = weekly active businesses receiving Nightly Close

Backend steps: 1. analytics queries + cohort SQL → 2. event ingestion table + writers → 3. admin endpoints.

Frontend screens: owner analytics section (charts, cohorts, comparisons, click-through) · internal admin panel

Done test: owner sees 38% of March customers returned in April; admin sees real MRR and activation funnel.

MODULE 14 — AI ASSISTANT

Features (all):

- One chat surface, two strictly separated abilities:
- **Help Assistant (RAG):** "how do I record credit?" → retrieve doc passages → answer ONLY from them → deep-link to screen → user's language → never invents features
- **Business Assistant (tool-calling):** "today's sales?" / "top product this week?" / "Ayesha's balance?" → Claude picks from 15-20 **read-only** functions — get_todays_sales, get_top_products, get_customer_balance, get_low_stock_items, get_review_summary, get_credit_outstanding, get_bookings_today, get_staff_sales, get_staff_performance, get_expenses_month, get_profit_month, get_campaign_stats, ... — executed server-side, hard-scoped to the authed business_id
- Hard rules: read-only v1 · every number from a tool result · "can't find that — check Reports" instead of guessing · ONE proactive useful note allowed (low-stock warning after top-products)
- AI review replies (M1) + what-if (M6) + monthly AI report (M13) share this infrastructure
- Later phase: same assistant over WhatsApp/SMS (owner messages their own system — no login) — capability no incumbent offers
- Cost control: fast cheap model, per-business rate limits, ~\$0.01/query class

Backend steps: 1. docs index + RAG endpoint → 2. tool registry + tool-calling loop (tenant-locked) → 3. streaming chat endpoint → 4. rate/cost caps.

Frontend: chat panel (streaming, language-matched, tappable links in answers)

Done test: "is hafte sab se zyada kya bika?" → exact real numbers + a stock warning; a made-up question gets an honest "not found."

MODULE 15 — REPORTS, DOWNLOADS & DATA EXPORT

Features (all):

- PDF reports (branded, language + currency): Monthly Business Report (sales, profit, top products, reviews, AI insights) · P&L statement · sales any range · staff performance/commissions · review/reputation report
- Excel/CSV: sales, customers, credit balances, stock, expenses
- Operational docs: invoices, receipts, quotations, QR posters, credit statements
- EVERY file: **Download + Send-by-WhatsApp** buttons
- Large exports = background jobs → notification + signed 24h link
- **Full account export** (zip) in Settings — "your data is never hostage"
- Permissions: staff = own sales only; full customer export = owner only (protects against staff stealing customer lists)

Backend steps: 1. HTML→Puppeteer report templates → 2. exceljs exporters → 3. export job queue + signed links → 4. account zip → 5. role gates.

Frontend: Reports screen — date-range picker · PDF cards · Excel chips · WhatsApp-send everywhere

Done test: accountant gets a correct P&L PDF on WhatsApp; a 2-year export arrives as a background link.

MODULE 16 — MOBILE APP + OFFLINE MODE

Features (all):

- React Native owner app (one codebase iOS + Android)
- Push notifications: new review, private complaint, order, booking
- Today's numbers · quick sale entry · customer lookup · AI Assistant
- **Offline mode:** sales entered without connectivity queue locally → auto-sync with conflict handling on reconnect (load-shedding proof — absent from Western

competitors)

Backend steps: 1. mobile-ready APIs → 2. push service → 3. offline sync protocol + conflict rules.

Mobile steps: app shell → push → quick sale → offline queue → AI chat → store releases.

Done test: sale in airplane mode syncs correctly later; 1★ complaint push arrives in seconds.

MODULE 17 — SEO & GROWTH ENGINE (complete SEO for everything)

Layer 1 — Noxtill's own acquisition SEO:

- Technical: SSR/SSG · unique titles/meta · Open Graph/Twitter cards · canonical URLs · auto XML sitemaps · robots.txt · schema.org (Organization, SoftwareApplication, Product+pricing, FAQPage, BreadcrumbList) · hreflang all launch languages · Core Web Vitals budgets in CI (LCP <2.5s, CLS <0.1) · image alt/optimization · mobile-first
- **Programmatic SEO:** auto-generated landing page per business type (200–300: "bakery management software", "clinic booking and billing system", "salon POS with reviews") from the M11 registry — unique useful copy, relevant widgets shown, interactive demo, FAQ schema, internal links; later type×country pages ("salon software UK"); new type row → new page, zero deploy
- Content engine: MDX blog (owner-problem guides: "reduce no-shows", "collect unpaid credit politely") with article schema + internal links

Layer 2 — customers' SEO (product value):

- Every collected review boosts the customer's local ranking
- Review widget adds fresh content (Review/AggregateRating schema) + "Powered by Noxtill" backlink
- Optional hosted mini-pages (menu/services/booking) with LocalBusiness/Menu/Service schema — "we make Google love your business," literally

Layer 3 — measurement:

- Search Console + analytics → internal SEO dashboard: rankings, indexed pages, signup attribution per landing page

Backend/frontend steps: 1. technical SEO on the marketing site → 2. page generator from types registry → 3. blog engine → 4. SEO dashboard → 5. hosted mini-pages.

SECTION C — CROSS-MODULE FLOWS (the connections that ARE the product)

1. Booking (M2) → confirmation + reminders (M-Engine) → completed → review request (M1) → 5★ Google / 1-3★ ticket → Nightly Close
2. Sale (M3) → stock (M5) + profit (M6) + customer record (M7) + payment/credit (M4) + audit — ONE transaction → review request → Nightly Close
3. Review data (M1) → CRM profile (M7) → segments (M9 campaigns) → analytics (M13)
4. Every module's numbers → widgets (M11) → Nightly Close → AI Assistant tools (M14) → reports (M15)
5. Business types (M11) → programmatic SEO pages (M17) → signups → analytics (M13)

SECTION D — SUPPORT SYSTEMS

- **Messaging Engine** (serves ALL modules): template registry per language/purpose (booking_confirm, booking_reminder, review_request, review_reminder, credit_reminder, order_status, owner_alert, nightly_close, birthday, campaign) · channel fallback (WA→SMS) · scheduling in business timezone · lifecycle queued→sent→delivered→read→failed · retries/backoff/dead-letter · quota gate + opt-out gate on every send
- **Nightly Close** (the signature): per-business scheduler at local close time → compose (v_daily_close + reviews + bookings + credit + low stock) → one message + day-view link + forward-to-partner link
- **Platform billing**: Stripe subscriptions, trial, quota enforcement, idempotent webhooks, regional gateway plugin interface, internal admin panel (M13)

SECTION E — 2-MONTH BUILD ORDER (what ships when)

Week	Build
1	Foundation (A8 full schema, auth, RBAC, tenant tests) + Messaging Engine · file Meta + Google + Stripe approvals Day 1
2-3	Module 1 (Reviews, complete)
3	Module 2 (Bookings, complete)

Week	Build
4	Module 3 (Sales/Billing) + Module 4 (Credit) + Home dashboard v1 + Nightly Close → <i>sellable</i>
5	Module 7 (CRM) + Module 5 (Inventory) + expenses
6	Module 6 (P&L + what-if) + Module 9 (Marketing)
7	Module 14 (AI Assistant) + Module 12 (Deep Search) + Module 15 (Reports/Exports)
8	Module 11 (Widgets + all types + AI fallback) + Module 8 (Staff) + billing/plans + onboarding wizard + marketing site + first 50 SEO pages (M17) → LAUNCH 🌙
Post-launch (architected now, built after)	Module 16 (Mobile + offline) · Module 10 (Multi-location) · WhatsApp AI assistant · social auto-post · Facebook/TripAdvisor sync · industry packs · remaining SEO pages · benchmarks (Year 2, only with 30–50+ businesses per segment, anonymized)

Slip rule: if any week slips, cut from Week 8 extras (customization UI, SEO page count) — NEVER from the core loop (sale → review → Nightly Close), security, or tenant isolation.

Deferred forever (documented so nothing is "missed"): AI legal/tax advice (liability — only generic date reminders with disclaimers, later) · AI "where to open a branch" external advice (no reliable data) · IoT beyond receipt printers/barcode scanners (Year 3 at earliest).